Decidability 02

Thumrongsak Kosiyatrakul
tkosiyat@cs.pitt.edu

Decidable Languages

E_{DFA}

Let $E_{\text{DFA}} = \{\langle A \rangle \mid A \text{ is a DFA and } L(A) = \emptyset\}$. Show that E_{DFA} is decidable.

- How do we know a DFA accepts no string?
  - This machine has no accept state ($F = \emptyset$) or
  - This machine has some accept states but are not reachable from the start state
- To show that E_{DFA} is decidable, we need a Turing machine T such that
  - If $\langle A \rangle \in E_{\text{DFA}}$, T accepts $\langle A \rangle$
  - If $\langle A \rangle \notin E_{\text{DFA}}$, T rejects $\langle A \rangle$

Not a Decider Example

- Consider this TM S:

  $S =$ "On input $\langle A \rangle$ where A is a DFA:

  1. For all strings w:
  2. Run TM M on input $\langle A, w \rangle$
  3. If M accepts $\langle A, w \rangle$, *reject*"

- Do you see the problem of TM S?
  - TM S is not a decider
  - Given $\langle A \rangle$ where A is a DFA and $L(A) = \emptyset$, S will run indefinitely
- For this problem:
  - We may have to play around with its state diagram
  - Create a Turing machine that analyze a DFA by marking all states reachable from the start state. If no accept state is marked, the DFA accepts no string

Decidable Languages

- Consider the following TM T:

  $T =$ "On input $\langle A \rangle$, where A is a DFA:

  1. Mark the start state of A.
  2. Repeat until no new states get marked:
  3. Mark any state that has a transition coming into it from any state that is already marked.
  4. If no accept state is marked, $accept$; otherwise, $reject$."

- We should not need to go down to the state diagram of a DFA unless it is necessary
  - Rely on graph theory
  - To prove, it must be based on graph theory

Decidable Languages

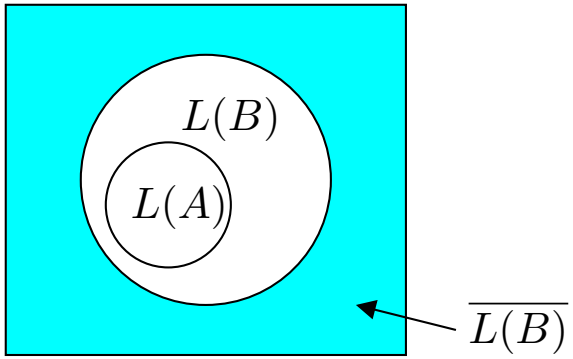
EQ_{DFA}

Let $EQ_{\text{DFA}} = \{\langle A, B\rangle \mid A \text{ and } B \text{ are DFAs and } L(A) = L(B)\}$.
Show that EQ_{DFA} is decidable.

- Given two DFAs A and B, is there any algorithm to check whether $L(A) = L(B)$?
- In other words, can we have a Turing machine F such that
  - If $\langle A, B\rangle \in EQ_{\text{DFA}}$ ($L(A) = L(B)$), F will accept $\langle A, B\rangle$ and
  - If $\langle A, B\rangle \notin EQ_{\text{DFA}}$ ($L(A) \neq L(B)$), F will reject $\langle A, B\rangle$
- Again, we cannot simply run both DFAs on all strings until one accept but the other reject
  - This may loop indefinitely
- Since $L(A)$ and $L(B)$ are sets, we need to use the set theory
  - $L(A) = L(B)$ iff $L(A) \subseteq L(B)$ and $L(B) \subseteq L(A)$

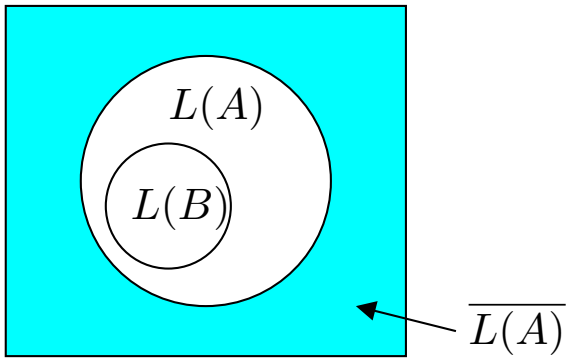
Decidable Languages

- $L(A) \subseteq L(B)$



From the above diagram $L(A) \cap \overline{L(B)} = \emptyset$

- $L(B) \subseteq L(A)$



From the above diagram $\overline{L(A)} \cap L(B) = \emptyset$

- $L(A) = L(B)$ iff $(L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B)) = \emptyset$.

The set of regular languages

- **Closed Under Complement**:
  - Given a DFA $A = (Q, \Sigma, \delta, q_0, F)$ we can construct a DFA A' where $L(A') = \overline{L(A)}$.
    - $A' = (Q, \Sigma, \delta, q_0, Q - F)$
- **Closed Under Union**:
  - Given two DFAs $A = (Q_A, \Sigma, \delta_A, q_A, F_A)$ and $B = (Q_B, \Sigma, \delta_B, q_B, F_B)$ we can construct a DFA $C = (Q_C, \Sigma, \delta_C, q_C, F_C)$ where $L(C) = L(A) \cup L(B)$.
    - $Q_C = Q_A \times Q_B$
    - $\delta_C((q_i, q_j), x) = (\delta_A(q_i, x), \delta_B(q_j, x))$ where $q_i \in Q_A$ and $q_j \in Q_B$.
    - $q_C = (q_A, q_B)$
    - $F_C = \{(q_i, q_j) \mid q_i \in F_A \text{ or } q_j \in F_B\}$.
- **Closed Under Intersection**:
  - Same as Closed Under Union except that
    - $F_C = \{(q_i, q_j) \mid q_i \in F_A \text{ and } q_j \in F_B\}$.

Decidable Language

- To show that

$$(L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B)) = \emptyset$$

  We need to construct a DFA C such that

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

  and check that $L(C) = \emptyset$ ($\langle C \rangle \in E_{\text{DFA}}$)
- This can be done by:
  1. Construct DFA D such that $L(D) = \overline{L(B)}$
  2. Construct DFA E such that $L(E) = L(A) \cap L(D)$
  3. Construct DFA F such that $L(F) = \overline{L(A)}$
  4. Construct DFA G such that $L(G) = L(F) \cap L(B)$
  5. Construct DFA C such that $L(C) = L(E) \cup L(G)$
  6. Run T (a decider for E_{DFA}) on input $\langle C \rangle$

Decidable Languages

- For simplicity, we can simply construct a DFA C such that

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

- Again, we need to show that C accepts no string ($L(C) = \emptyset$)
- This is an example of a TM:

$F =$"On input $\langle A, B \rangle$, where A and B are DFAs:

1. Construct DFA C such that

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

2. Run TM T from Theorem 4.4 on input $\langle C \rangle$.
3. If T accepts $\langle C \rangle$, *accept*. If T rejects $\langle C \rangle$, *reject*.

Prove that EQ_{DFA} is Decidable

Prove that $EQ_{\mathrm{DFA}} = \{\langle A, B\rangle \mid A \text{ and } B \text{ are DFAs and } L(A) = L(B)\}$ is decidable.

Solution: Construct a TM F as follows:

$F =$"On input $\langle A, B\rangle$, where A and B are DFAs:

1. Construct DFA C such that $L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$
2. Run TM T from Theorem 4.4 on input $\langle C\rangle$.
3. If T accepts $\langle C\rangle$, *accept*. If T rejects $\langle C\rangle$, *reject*.

Now, we need to prove that F is a decider for EQ_{DFA}.

1. Assume that $\langle A, B\rangle \in EQ_{\mathrm{DFA}}$. Since $\langle A, B\rangle \in EQ_{\mathrm{DFA}}$, A and B are DFAs and $L(A) = L(B)$. Since $L(A) = L(B)$, according to the definition of the language of DFA C, $L(C) = \emptyset$. Since $L(C) = \emptyset$, $\langle C\rangle \in E_{\mathrm{DFA}}$. By running T on input $\langle C\rangle$, T will accept $\langle C\rangle$. Since T accepts $\langle C\rangle$, F accepts $\langle A, B\rangle$.
2. Assume that $\langle A, B\rangle \notin EQ_{\mathrm{DFA}}$. Since $\langle A, B\rangle \notin EQ_{\mathrm{DFA}}$, A and B are DFAs and $L(A) \neq L(B)$. Since $L(A) \neq L(B)$, according to the definition of the language of DFA C, $L(C) \neq \emptyset$. Since $L(C) \neq \emptyset$, $\langle C\rangle \notin E_{\mathrm{DFA}}$. By running T on input $\langle C\rangle$, T will reject $\langle C\rangle$. Since T rejects $\langle C\rangle$, F rejects $\langle A, B\rangle$.

This shows that F is a decider for EQ_{DFA}. Therefore, EQ_{DFA} is decidable.

Decidability

- So far we have the following Turing machines:
  1. TM M that decides A_{DFA} where

     $$A_{\text{DFA}} = \{\langle B, w\rangle \mid B \text{ is a DFA that accepts } w\}$$

  2. TM N that decides A_{NFA} where

     $$A_{\text{NFA}} = \{\langle B, w\rangle \mid B \text{ is an NFA that accepts } w\}$$

  3. TM P that decides A_{REX}

     $$A_{\text{REX}} = \{\langle R, w\rangle \mid R \text{ is a regular expression that generates } w\}$$

  4. TM T that decides E_{DFA}

     $$E_{\text{DFA}} = \{\langle A\rangle \mid A \text{ is an DFA and } L(A) = \emptyset\}$$

  5. TM F that decides EQ_{DFA}

     $$EQ_{\text{DFA}} = \{\langle A, B\rangle \mid A \text{ and } B \text{ are DFAs and } L(A) = L(B)\}$$

- We can use these TMs as helper machines

ONE_{DFA}

- How do we know that a DFA accepts exactly one string?
- This problem corresponds with the language ONE_{DFA} as follows:

$$ONE_{\text{DFA}} = \{\langle A \rangle \mid A \text{ is a DFA and } |L(A)| = 1\}$$

- How to show that ONE_{DFA} is decidable?
  - Construct a TM M' such that
    1. If $\langle A \rangle \in ONE_{\text{DFA}}$, M' accepts $\langle A \rangle$
    2. If $\langle A \rangle \notin ONE_{\text{DFA}}$, M' rejects $\langle A \rangle$
- Note that we cannot run a DFA A on all string until it accepts a string because it may loop indefinitely
- But if $L(A) \neq \emptyset$, it will not loop indefinitely
- So, we need to check that the language of a given DFA is not empty first

- Construct a TM M' as follows:

  $M' =$ "On input $\langle A \rangle$ where A is a DFA:

  1. Run T on input $\langle A \rangle$
  2. If T accepts $\langle A \rangle$, *reject*.
  3. If T rejects $\langle A \rangle$, run A on all inputs until it accepts an input string w
  4. Construct a DFA B such that $L(B) = \{w\}$
  5. Run F on input $\langle A, B \rangle$.
  6. If F accepts $\langle A, B \rangle$, *accept*. Otherwise, *reject*."

- Prove that if $\langle A \rangle \in ONE_{\text{DFA}}$, M' accepts $\langle A \rangle$:

  Assume that $\langle A \rangle \in ONE_{\text{DFA}}$. Since $\langle A \rangle \in ONE_{\text{DFA}}$, A is a DFA and $|L(A)| = 1$. Since $|L(A)| = 1$, $L(A) \neq \emptyset$. Since $L(A) \neq \emptyset$, $\langle A \rangle \notin E_{\text{DFA}}$. Since T is a decider for E_{DFA}, by running T on input $\langle A \rangle$, T will reject $\langle A \rangle$. Since $|L(A)| = 1$, by running A on all strings, it will eventually accept a string w. Since $|L(A)| = 1$ and A accepts w, $L(A) = \{w\}$ which is the same as the language of B. Since $L(A) = L(B)$, $\langle A, B \rangle \in EQ_{\text{DFA}}$. By running F on input $\langle A, B \rangle$, F will accept $\langle A, B \rangle$. Since F accepts $\langle A, B \rangle$, M' accepts $\langle A \rangle$.

- Prove that if $\langle A \rangle \notin ONE_{\text{DFA}}$, M' rejects $\langle A \rangle$:

  Assume that $\langle A \rangle \notin ONE_{\text{DFA}}$. Since $\langle A \rangle \notin ONE_{\text{DFA}}$, A is a DFA and $|L(A)| \neq 1$. In other words, $L(A) = \emptyset$ or $|L(A)| > 1$.

  If $L(A) = \emptyset$, $\langle A \rangle \in E_{\text{DFA}}$. Since T is a decider for E_{DFA}, by running T on input $\langle A \rangle$, T will accept $\langle A \rangle$. Since T accepts $\langle A \rangle$, M' rejects $\langle A \rangle$.

  If $|L(A)| > 1$, by running A on all strings, it will eventually accept a string w. Since $|L(A)| > 1$ and A accepts w, $L(A) \neq \{w\}$ which is not the same as the language of B. Since $L(A) \neq L(B)$, $\langle A, B \rangle \notin EQ_{\text{DFA}}$. By running F on input $\langle A, B \rangle$, F will reject $\langle A, B \rangle$. Since F rejects $\langle A, B \rangle$, M' rejects $\langle A \rangle$.

This shows that M' is a decider for ONE_{DFA}. Therefore, ONE_{DFA} is decidable.